Replicating Intel's Acoustic Weld-Defect Detection Baseline at 16 kHz

Jaden Majid Hieu Le Chris Tardy Irene Berezin
Grad Student Descent · AI 360
Experiment snapshot: November 4, 2026

Abstract

We reimplement Intel's one-dimensional convolutional autoencoder for acoustic weld-defect detection using the released mono 16 kHz recordings. The experiment retains the published best bottleneck and approximately preserves its spectral analysis duration, while making unspecified implementation choices explicit. The designated primary run achieves test ROC-AUC 0.8178, compared with the published 192 kHz result of 0.8460. Across three completed 50-epoch runs, mean test ROC-AUC is 0.8180 ± 0.0007 and mean validation ROC-AUC is 0.8158 ± 0.0005 (sample standard deviations). These results establish a baseline for the released audio, but do not isolate the effect of sampling rate or establish exact reproduction of the original benchmark.

1 Scope and data

Stemmer et al. [1] detect defective welds by reconstructing normal welding audio. We reproduce their audio-only model from scratch as a baseline for subsequent representation-learning experiments. This is a single configuration with three planned random seeds, rather than a repeat of the paper's FFT/bottleneck grid search. The configuration is chosen from the paper, before inspecting replication test scores.

The audited release contains 4,040 mono FLAC recordings at 16 kHz, spanning normal welds and eleven defect categories. We retain the manifest partitions (Table 1); only normal training recordings enter optimization. Recordings last 25–38 s. Validation contains labeled defects and is used to select epochs, so training is normal-only but selection uses anomaly labels. The audit checks split counts, unique recording identifiers, waveform validity, and file hashes. No new split, trimming, or augmentation is applied.

Partition	Normal	Defective	Total
Training	576	0	576
Validation	122	1,610	1,732
Test	121	1,611	1,732

Table 1: Manifest partitions, matching the published protocol. Test labels are reserved for final evaluation.

2 Method and implementation choices

Features. The paper's selected FFT length is 16,384 at 192 kHz, corresponding to 85.33 ms. Scaling by sampling rate and rounding to an even integer gives $N = 1{,}366$ at 16 kHz (85.375 ms), with hop $H = 683$ (42.6875 ms). We compute a one-sided Hann-window STFT without centering or normalization, then $X_{ft} = \log\max(|\,\mathrm{STFT}(x)_{ft}|, 10^{-8})$, giving $F = 684$ frequency bins. Duration is preserved approximately; bandwidth is reduced to 8 kHz and the frequency dimension changes.

Architecture. Non-affine BatchNorm standardizes frequency channels. The encoder channel sequence is $684 \to 1024 \to 1024 \to 1024 \to 1024 \to 48$; five transposed convolutions reverse this sequence. All kernels have size three, stride one, and no padding. The encoder reduces time by ten frames and the decoder restores it. Activations are LeakyReLU with slope 0.01, except for a scalar output PReLU initialized to 0.25. The model has 23,380,701 trainable parameters. BatchNorm statistics update only during training; its standardized output, detached from the computation graph, is the reconstruction target.

Chunking, objective, and score. Inputs are divided into 192-frame chunks, with a final end-aligned chunk when needed. Short chunks are reflect-padded. Padding is masked out of the loss and score, although it influences convolutions and training BatchNorm. Waveforms shorter than one FFT window are reflect-padded; incomplete trailing STFT windows are discarded. For chunk c, let Z_c be the standardized target, $\widehat{Z}_c$ its reconstruction, and m_{ct} indicate valid frames. We minimize

$$\mathscr{L} = \frac{\sum_{c,t,f} m_{ct} (\widehat{Z}_{cft} - Z_{cft})^2}{F \sum_{c,t} m_{ct}}. \tag{1}$$

At inference, squared errors are averaged across frequency and, for frames covered by multiple chunks, across their reconstructions. The recording score is the mean over its original frames. Higher scores indicate anomalies. Thus, overlap does not give the final frames extra weight in recording scores.

3 Experimental protocol and observed results

We train in float32 for 50 epochs with batch size 16, Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$), and no weight decay. A cosine one-cycle schedule peaks at 10^{-4} after a 30% warmup, with initial division factor 25 and final division factor 10,000; Adam momentum is fixed. Training uses 2,880 chunks, shuffled with the run seed, and validation uses 8,580 chunks. Runs use seeds 42, 43, and 44 on an NVIDIA GeForce RTX 5070 Ti with PyTorch 2.14.1+cu130. Deterministic PyTorch algorithms are enabled.

For each seed, the checkpoint with maximum pooled validation ROC-AUC is retained; ties retain the earlier epoch. Seed 42 is the designated primary result and is not replaced by the best seed. After all runs finish, checkpoint hashes are frozen in `selection.json` before test evaluation is enabled. The primary metric pools all defects versus normal recordings. Per-defect ROC-AUC compares each defect with the same normal validation recordings; their unweighted mean is a separate macro metric.

Experiment	Epochs done	Best epoch	Val. AUC	Test AUC
Published, 192 kHz	50	—	0.8451	0.8460
Replication, seed 42	50/50	47	0.8155	0.8178
Replication, seed 43	50/50	40	0.8155	0.8174
Replication, seed 44	50/50	49	0.8164	0.8188
Mean ± sample SD	—	—	0.8158 ± 0.0005	0.8180 ± 0.0007

Table 2: Validation-selected results. Published values are from Tables 7 and 9 of [1]. All three replication runs and evaluations are complete; test scores use the validation-selected, frozen checkpoints. The mean summarizes individual runs, without ensembling.

The primary test ROC-AUC is 0.0282 below the published test value; its validation ROC-AUC is 0.0296 below the published validation value. Across three seeds, the test sample standard deviation is 0.0007; this measures variation from initialization and training order, rather than uncertainty from sampling new recordings. Seed 42's test macro defect AUC is 0.8497. Its highest per-defect test AUCs are overlap (0.9993), crater cracks (0.9882), and undercut (0.9839); warping (0.6162) and excessive penetration (0.6261) remain difficult. We recomputed metrics from saved predictions,

checked validation results against run summaries and histories, and verified that test provenance and checkpoint hashes match the frozen selection.

An independent real-data smoke check reduced MSE from 1.0001 to 0.2016 over 100 updates on a fixed batch from two normal training recordings and verified checkpoint reload agreement. This checks optimization and serialization, without measuring anomaly-detection generalization. The completed runs each record approximately 7.3 minutes of run time; data preparation and the smoke check are excluded.

4 Limitations and reproducibility

This experiment adapts a published architecture to a different input release. Preserving STFT duration does not restore frequencies above 8 kHz, and the smaller frequency axis also changes parameter count. BatchNorm behavior, padding, activation details, chunk length, and optimizer scheduling resolve omissions in the source paper. Consequently, the validation gap cannot be attributed to bandwidth alone. No ablation isolates these choices. Selection across 50 validation epochs can also favor validation-specific behavior. No confidence interval, streaming latency measurement, or robustness to a new welding station is established here. The source dataset also has acknowledged label noise [1].

The implementation is in `src/intel_baseline/`; the full settings are in `configs/intel_baseline.json`. After installing the environment and caching the dataset archive, the repository-root command below audits the data, runs or resumes training, freezes selection, evaluates the test partition, and generates the report:

```
bash scripts/run_intel_baseline.sh
```

Artifacts under `artifacts/intel_baseline/` retain settings, histories, best/latest checkpoints, predictions, and provenance. Random-generator, optimizer, and scheduler states are saved for epoch-boundary resumption. The dataset audit records revision `58e58e59beb7` and fingerprint prefix `d250a3293b81d849`; full identifiers are in `data/intel_baseline/audit.json`. Run metadata records code commit `0c2fba904be5` with a dirty working tree; that commit alone therefore does not fully specify the executed source. All three seed runs and frozen-checkpoint test evaluations are complete in this artifact snapshot.

References

[1] Georg Stemmer, Jose A. Lopez, Juan A. Del Hoyo Ontiveros, Arvind Raju, Tara Thimmanaik, and Sovan Biswas. Unsupervised Welding Defect Detection Using Audio and Video. *arXiv preprint arXiv:2409.02290*, 2024. https://arxiv.org/abs/2409.02290.